

YatraLink — Technical Design Document

Version 1.0 • System Architecture & Engineering Design

1. Architecture Principles

- Deterministic railway logic first; AI is an explanation layer, not the source of truth for availability.
- Separate ingestion, availability, route generation and ranking so each is independently testable.
- Cache read-heavy railway data while making freshness explicit.
- Keep booking behind an adapter for future authorized integration.
- Recommendations must be reproducible from their data snapshot.

2. Logical Architecture

Web/PWA → API Gateway → Search Orchestrator → Train Data Service + Availability Service + Connection Engine + Ranking Engine → PostgreSQL/Redis. An AI Explanation Service consumes validated journey options. A future Booking Adapter is isolated behind a provider interface.

3. Main Components

Component	Responsibility
Next.js client	Search UI, results, filters, journey details.
FastAPI API	Validation, orchestration and API contracts.
Train Data Service	Stations, trains, route sequences, schedules.
Availability Service	Fetch/cache availability and fares.
Connection Engine	Generate and validate candidates.
Ranking Engine	Score and sort valid journeys.
Redis	Cache, rate limiting, short-lived state.
PostgreSQL/Supabase	Persistent application/reference data.
AI Explanation Service	Explain validated options without inventing facts.
Booking Adapter	Future authorized booking provider integration.

4. Data Model

Entity	Important fields
stations	id, code, name, city, state, lat, lon
trains	number, name, type, active
train_routes	train_id, station_id, sequence, arrival, departure, day_offset
train_classes	train_id, class_code
availability	date, train, from, to, class, quota, status, updated_at

Entity	Important fields
fares	train, from, to, class, quota, amount, updated_at
searches	session/user, origin, destination, date, passengers, class, preferences
journey_options	search_id, score, total_fare, duration, risk, changes
journey_segments	journey_id, train, from, to, departure, arrival, availability

5. Connection Engine

Represent stations as nodes and train segments as time-dependent edges. A segment is valid only if the train serves origin and destination in the correct sequence and inventory is acceptable. A multi-train connection is valid only when the next departure occurs after the previous arrival plus the configured minimum transfer buffer.

6. Candidate Generation

- Generate direct candidates first.
- Generate same-train split candidates using intermediate stations.
- Generate one-change candidates by matching arrival stations to later departures.
- Generate two-change candidates only when needed.
- Prune candidates beyond configured duration, budget or change limits.
- Avoid cycles and duplicate stations.
- Deduplicate equivalent journey patterns.

7. Candidate Validation

- Chronological validity.
- Station identity and transfer compatibility.
- Train route sequence validity.
- Availability state validation.
- Passenger/class/quota compatibility.
- Connection buffer validation.
- Station-transfer penalty.
- Data freshness check.

8. Ranking Model

Initial conceptual score = availability + convenience + time + price + continuity – transfer risk – station-transfer risk – excessive waiting. Weights should be configurable, versioned and later tuned using observed behavior.

9. Search Pipeline

Validate request → normalize stations → load relevant routes → query/cache availability → generate candidates → validate → calculate price/time/risk → score → deduplicate → return top options with explanation metadata.

10. API Sketch

Method	Endpoint	Purpose
POST	/api/v1/searches	Create journey search.
GET	/api/v1/searches/{id}	Retrieve results.
GET	/api/v1/stations?q=	Station autocomplete.
GET	/api/v1/trains/{number}	Train route/details.
POST	/api/v1/journeys/validate	Revalidate selected journey.
POST	/api/v1/booking/quote	Future booking quote adapter.
POST	/api/v1/booking/create	Future authorized booking adapter.

11. Reliability & Failure Handling

- Provider timeout → use cached data only if freshness permits and label it.
- Provider unavailable → return UNKNOWN rather than guessing.
- Partial data → mark affected segments unknown.
- Search explosion → enforce candidate/station limits.
- Duplicate results → canonicalize segments.
- Booking revalidation → recheck immediately before future booking action.

12. Security

- Server-side secrets only.
- Strong authorization/RLS where applicable.
- Rate limiting and abuse controls.
- Minimal passenger-data storage.
- Audit logs for sensitive operations.
- No CAPTCHA bypass or unauthorized railway-account automation.

13. Testing Strategy

Layer	Tests
Unit	Time calculations, route validity, scoring, station matching.
Integration	Provider adapters, DB queries, cache behavior.
Property-based	No negative travel time, no cycles, chronological consistency.
End-to-end	Search → results → journey detail.
Load	Concurrent search and cache behavior.
Regression	Known difficult routes and edge cases.

14. Deployment

Recommended initial deployment: managed Next.js hosting, managed FastAPI/container hosting, Supabase PostgreSQL, managed Redis, GitHub Actions CI/CD, environment-managed secrets and centralized logs/metrics.

15. Evolution

V1 discovery → V2 live availability and alerts → V3 authorized booking → V4 personalized travel assistant and mobile applications.